

Industrial Internship Report on " File Organizer using Python"

Prepared by
[Surisetty Indu Varshini]

Executive Summary

This report presents the details of the Industrial Internship provided by Upskill Campus in collaboration with its Industrial Partner UniConverge Technologies Pvt. Ltd. (UCT). The internship was designed to provide practical exposure to real-world problem solving through the application of Python programming concepts.

The internship was centered around a project-based learning approach, where participants were required to design and implement a solution within the stipulated duration. The project assigned for this internship focused on developing a File Organizer application using Python.

My project involved designing and implementing a Python-based File Organizer that scans a user-selected directory, categorizes files based on their types (such as images, documents, music, and videos), and automatically organizes them into respective folders. The application also included a graphical user interface (GUI) developed using Tkinter to enhance user interaction and usability.

This internship provided a valuable opportunity to apply theoretical programming knowledge to a practical use case. It helped strengthen my understanding of Python libraries, file handling mechanisms, and GUI development. Overall, the internship experience was highly enriching and contributed significantly to my technical skills and problem-solving abilities.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About Project Modules / System Components.....	4
2.2	Objective of the Project	5
2.3	Scope of the Project	6
2.4	Tools and Technologies Used.....	6
2.5	Reference.....	7
2.6	Glossary.....	7
3	Problem Statement.....	8
4	Existing and Proposed solution.....	10
5	Proposed Design/ Model.....	11
5.1	High Level Diagram (if applicable).....	11
5.2	Low Level Diagram (if applicable).....	12
5.3	Interfaces (if applicable).....	12
6	Performance Test.....	13
6.1	Test Plan/ Test Cases.....	16
6.2	Test Procedure	17
6.3	Performance Outcome.....	17
7	My learnings.....	18
8	Future work scope.....	19

1 Preface

This internship provided an excellent opportunity to gain practical exposure to real-world application development using Python. Over the course of the internship, consistent efforts were made to understand core programming concepts, apply problem-solving techniques, and develop a functional software solution.

The six-week internship journey involved learning, designing, implementing, and refining a Python-based project titled “**File Organizer using Python.**” The project was focused on solving a common real-world problem of managing and organizing files within a directory. The application was designed to scan a user-selected folder, identify files based on their extensions, categorize them into predefined groups such as Images, Documents, Music, and Videos, and automatically move them into their respective folders. A graphical user interface (GUI) was also developed using Tkinter to improve user interaction and usability.

Participating in this internship was highly beneficial for career development, as it enabled the practical application of theoretical knowledge. The experience helped build a deeper understanding of Python programming, file handling mechanisms, automation logic, and GUI design. It also strengthened problem-solving abilities, debugging skills, and confidence in developing real-world applications.

The opportunity provided by Upskill Campus and UniConverge Technologies Pvt. Ltd. allowed for structured learning and guided project execution. The program was well-planned, with a clear focus on progressive skill development, starting from fundamental concepts and gradually moving toward project implementation and refinement.

Throughout the internship, valuable technical and practical skills were acquired, including working with Python libraries such as **os**, **shutil**, and **tkinter**, understanding directory operations, implementing logical conditions, and designing user-friendly interfaces. Overall, the internship experience was enriching, informative, and instrumental in enhancing technical competence.

I would like to express my sincere gratitude to **Upskill Campus, UniConverge Technologies Pvt. Ltd.**, and the mentors and coordinators who provided guidance and support throughout the internship. Their assistance and structured approach greatly contributed to the successful completion of the project.

Finally, I encourage my juniors and peers to actively participate in internships and practical projects, as they provide invaluable learning experiences, improve industry readiness, and help bridge the gap between academic knowledge and real-world application.

2 Introduction

2.1 Project Modules / System Components

The File Organizer using Python is a software-based automation application designed to simplify file management within a directory. The system is built using Python and leverages standard libraries to perform file scanning, classification, and organization tasks. The project consists of the following key components:

2.1.1 i. File Scanning Module

The File Scanning Module is responsible for reading the contents of a user-selected directory. It retrieves the list of files present in the folder and identifies valid files for processing. This module uses Python's **os library** to access directory paths and file information.

- It scans the selected directory and detects all available files
- It differentiates files from folders
- It prepares files for classification based on extensions

2.1.2 ii. File Classification Module

The File Classification Module categorizes files based on their extensions. Different file types such as images, documents, music, and videos are identified and grouped accordingly. This module ensures that files are assigned to the correct categories.

- It identifies file types using extensions (e.g., .jpg, .pdf, .mp3, .mp4)
- It maps files into predefined categories
- It handles unsupported or unknown file types

2.1.3 iii. Folder Creation & Organization Module

This module ensures that appropriate folders exist before moving files. If category folders such as Images, Documents, Music, or Videos are missing, the system automatically creates them. Python's **os and shutil libraries** are used for folder management and file movement.

- It creates category folders dynamically
- It moves files into respective folders
- It prevents file duplication or errors

2.1.4 iv. Graphical User Interface (GUI) Module

The GUI Module provides a simple and interactive interface for the user. Built using Python's **Tkinter library**, it allows users to select the folder they wish to organize without manually entering directory paths.

- It enables folder selection via a dialog window
- It improves user experience and accessibility
- It provides success notifications

2.2 Objective of the Project

The primary objective of this project is to develop a Python-based File Organizer application that automates the process of managing and arranging files within a directory. The project aims to reduce the manual effort required to sort files by automatically scanning a user-selected folder, identifying file types based on their extensions, and organizing them into appropriate categories such as Images, Documents, Music, and Videos.

Another key objective is to design a simple and user-friendly graphical user interface (GUI) using Tkinter, enabling users to select directories easily without needing technical knowledge of file paths. The project also seeks to demonstrate the practical application of Python programming concepts, including file handling, directory management, and automation techniques.

Overall, the project focuses on improving file management efficiency, minimizing user effort, and providing a structured approach to organizing digital files.

2.3 Scope of the Project

The scope of this project is to design and develop a Python-based File Organizer application that automates the process of sorting and managing files within a directory. The application is intended to scan a user-selected folder, identify files based on their extensions, and categorize them into predefined groups such as Images, Documents, Music, Videos, and Others.

The project includes the implementation of file handling mechanisms to read directory contents, detect file types, create necessary folders dynamically, and move files into their respective

locations. A graphical user interface (GUI) is also incorporated using Tkinter to allow users to select directories conveniently, improving usability and accessibility.

This project primarily focuses on basic file organization functionality and supports common file formats. Advanced features such as cloud integration, duplicate detection, file compression, and scheduling automation are beyond the current scope but can be considered for future enhancements.

Overall, the project scope is limited to demonstrating practical Python programming, directory management, file classification, and GUI-based interaction for efficient file organization.

2.4 Tools and Technologies Used

The File Organizer application was developed using Python and its standard libraries to implement file management and graphical user interface functionality. The following tools and technologies were used during the project development:

- **Python:**

Python served as the primary programming language for developing the application. It was used to implement the file scanning, classification, folder creation, and file organization logic.

- **OS Library:**

The `os` module was used for interacting with the operating system, accessing directory paths, retrieving file lists, checking file types, and managing folders.

- **Shutil Library:**

The `shutil` module was utilized for performing high-level file operations such as moving files between directories and managing file organization tasks.

- **Tkinter:**

Tkinter was used to design the graphical user interface (GUI) of the application. It enabled user interaction by providing a window-based interface for selecting directories and displaying status messages.

- **Code Editor / IDE:**

A Python-supported development environment (such as IDLE / Visual Studio Code) was used to write, edit, and execute the program.

- **Operating System:**

The application was developed and tested on a Windows-based environment for directory operations and file management.

2.5 Reference

- [1] Python Software Foundation, *Python Official Documentation* – <https://docs.python.org/>
- [2] Tkinter Documentation, *Python GUI Programming with Tkinter* – <https://docs.python.org/3/library/tkinter.html>
- [3] Online Python Tutorials and Examples for File Handling Concepts

2.6 Glossary

Terms	Acronym
Graphical User Interface	GUI
File Organizer	FO
Operating System	OS
Python Standard Library	PSL
File Extension	FE

3 Problem Statement

In everyday computer usage, directories such as Downloads, Desktop, and Documents often contain a large number of files of different types stored together without any structured organization. As files accumulate over time, it becomes increasingly difficult for users to locate specific documents, media files, or resources. Manual sorting of files into appropriate folders is time-consuming, repetitive, and prone to errors, especially when dealing with numerous files.

Users frequently face challenges in maintaining a clean and organized file system, which can reduce productivity and lead to inefficient file management. There is therefore a need for an automated solution that can simplify the process of file organization and reduce manual effort.

To address this problem, the project proposes the development of a Python-based File Organizer application capable of scanning a selected directory, identifying files based on their extensions, categorizing them into predefined groups, and automatically arranging them into appropriate folders. This solution aims to improve efficiency, save time, and provide a structured approach to file management.

4 Existing and Proposed solution

Provide summary of existing solutions provided by others, what are their limitations?

What is your proposed solution?

What value addition are you planning?

4.1.1 Existing Solutions and Their Limitations

File organization is traditionally performed manually by users, where files are sorted into folders based on their types. While this method is simple, it has several limitations. Manual sorting is time-consuming, repetitive, and inefficient, especially when dealing with large numbers of files. Users may forget to organize files regularly, leading to cluttered directories. Additionally, human errors such as misplacing files or creating inconsistent folder structures are common.

Some advanced file management tools exist, but they often require installation, complex configurations, or paid subscriptions. These tools may also include unnecessary features that increase system overhead and reduce usability for basic file organization needs.

4.1.2 Proposed Solution

To overcome the limitations of existing methods, this project proposes a **Python-based File Organizer application** that automates the file sorting process. The application scans a user-selected directory, identifies files based on their extensions, categorizes them into predefined groups such as Images, Documents, Music, Videos, and Others, and automatically moves them into respective folders.

The system also incorporates a simple graphical user interface (GUI) using Tkinter, allowing users to select directories easily without requiring technical knowledge of file paths.

4.1.3 Value Addition of the Proposed Solution

The proposed solution offers several advantages over traditional approaches. It eliminates the need for manual file sorting, saving time and reducing user effort. The application dynamically creates folders when needed, ensuring consistent organization. The GUI-based interaction improves accessibility and usability. Furthermore, the solution is lightweight, requires no external dependencies, and leverages Python's standard libraries, making it efficient and easy to maintain.

Overall, the project provides a practical, user-friendly, and automated approach to file management.

4.2 Code submission (Github link)

<https://github.com/Indu-varshini/upskillcampus/blob/main/File%20Organizer%20using%20Python.py>

4.3 Report submission (Github link) :

https://github.com/Indu-varshini/upskillcampus/blob/main/File%20Organizer%20using%20Python_Indu%20Varshini_USC_UCT.pdf

5 Proposed Design/ Model

The proposed system automates file organization through a sequence of operations starting from user input to final structured output. The workflow includes folder selection via GUI, directory scanning, file classification using extensions, automatic folder creation, and file relocation.

5.1 High Level Diagram (if applicable)

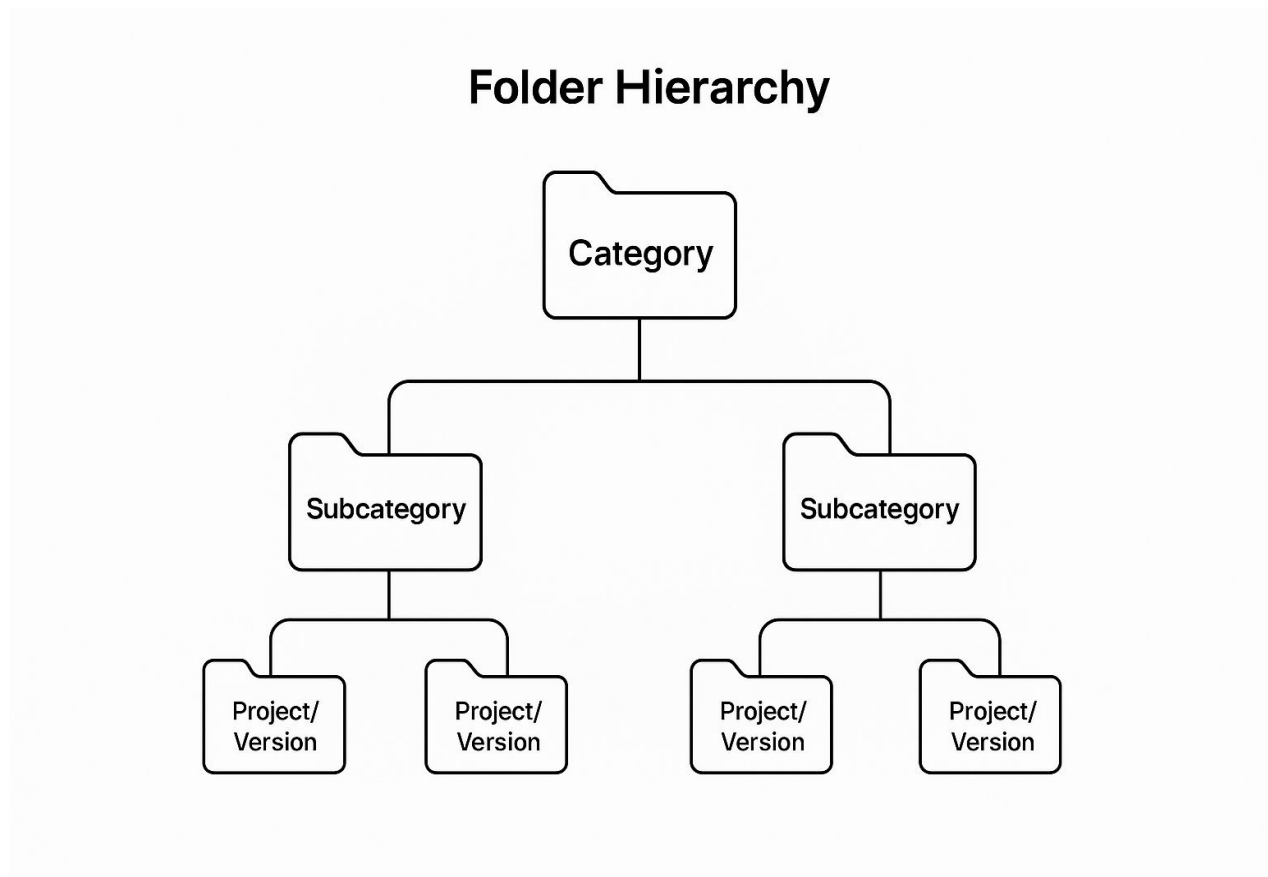


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

The process begins when the user selects a folder through the graphical user interface. The system scans the directory, identifies file types, creates category folders if required, and moves files into appropriate locations. The workflow ends with a completion notification.

5.2 Low Level Diagram (if applicable)

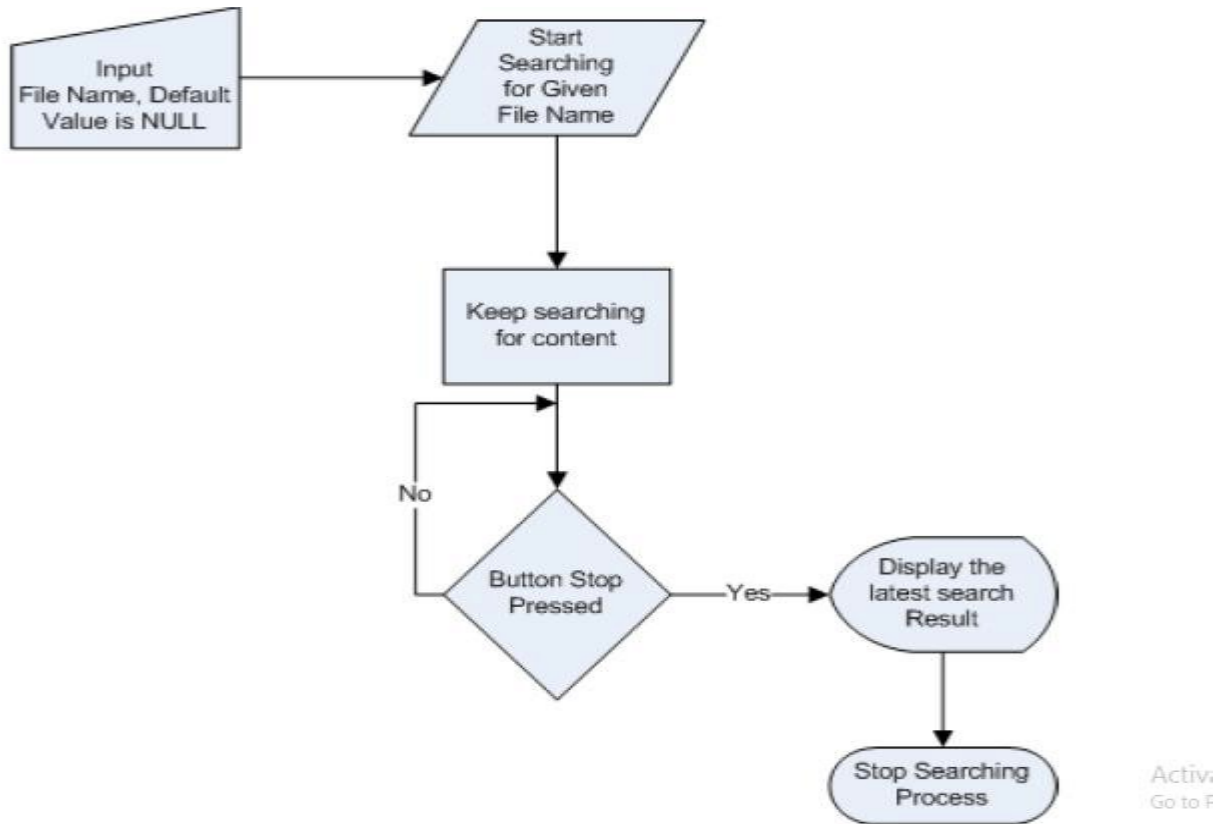


Figure 2: LOW LEVEL DIAGRAM OF THE SYSTEM

Each file is examined individually. The system checks whether the item is a valid file, extracts its extension, compares it against predefined categories, and moves it accordingly. Files without matching categories are redirected to the “Others” folder.

5.3 Interfaces (if applicable)

The user interface is implemented using Tkinter, allowing directory selection through a dialog window. The backend interacts with the operating system using Python libraries to read, classify, and relocate files, forming a continuous data flow.

6 Performance Test

Performance testing for the File Organizer application was conducted to evaluate the efficiency and reliability of the system under practical usage conditions. Since the application performs file system operations, the primary constraints considered were execution speed, memory usage, and correctness of file classification.

Identified Constraints

The following constraints were identified for the system:

- **Execution Time:**

The application must process directories containing multiple files without noticeable delay.

- **Memory Usage:**

The system should operate efficiently without consuming excessive memory resources, even when scanning folders with many files.

- **File Classification Accuracy:**

Files must be correctly categorized based on their extensions to ensure proper organization.

- **File Handling Reliability:**

The program must avoid data loss or file corruption during movement operations.

Handling of Constraints in Design

The application design was structured to address these constraints effectively:

- Standard Python libraries (`os`, `shutil`) were used to ensure efficient file operations.
- Files are processed sequentially to minimize memory overhead.
- Extension-based classification logic ensures consistent categorization.
- Folder creation uses dynamic checks to avoid redundant operations.
- Built-in file movement functions reduce the risk of corruption.

Testing and Observations

The application was tested using sample directories containing mixed file types such as images, documents, audio, and video files.

- The program executed successfully without runtime errors.
- Files were organized accurately into respective folders.
- Execution time remained minimal for typical folder sizes.
- No file corruption or data loss was observed.

Impact of Constraints and Recommendations

While the application performed efficiently for normal usage scenarios, certain constraints may impact performance in large-scale environments:

- Very large directories with thousands of files may increase execution time.
- Systems with limited hardware resources may experience slower processing.

To address these potential issues, future improvements may include:

- Batch processing mechanisms
- Progress indicators for user feedback
- Optimized classification algorithms

6.1 Test Plan/ Test Cases

The File Organizer application was tested under various scenarios to verify correctness, reliability, and functional behavior. The objective of testing was to ensure accurate file classification, proper folder creation, and safe file movement.

1.Test Case 1: Folder Selection Validation

Objective:

To verify that the user can successfully select a directory using the GUI.

Input:

User selects a valid folder through the Tkinter dialog.

Expected Result:

Selected folder path is accepted and processing begins.

Outcome:

Folder selection works correctly without errors.

2.Test Case 2: Mixed File Type Handling**Objective:**

To ensure files of different types are categorized properly.

Input:

Folder containing .jpg, .pdf, .mp3, .mp4, .txt files.

Expected Result:

Files are moved into Images, Documents, Music, Videos folders.

Outcome:

Files categorized and moved successfully.

3.Test Case 3: Unsupported File Types**Objective:**

To validate handling of unknown file extensions.

Input:

Files with extensions not defined in categories (e.g., .xyz).

Expected Result:

Files moved into “Others” folder.

Outcome:

Unsupported files handled correctly.

4.Test Case 4: Empty Folder Handling**Objective:**

To verify system behavior when folder contains no files.

Input:

Empty directory selected by user.

Expected Result:

No errors; program completes gracefully.

Outcome:

Program executed without issues.

5.Test Case 5: Folder Creation Verification**Objective:**

To ensure category folders are created automatically.

Input:

Directory without Images/Documents/Music/Videos folders.

Expected Result:

Required folders created dynamically.

Outcome:

Folders created successfully.

6.Test Case 6: File Safety Check**Objective:**

To confirm files are not corrupted during movement.

Input:

Various file types moved by application.

Expected Result:

Files remain accessible and intact.

Outcome:

No file corruption observed.

6.2 Test Procedure

The testing of the File Organizer application was carried out using a structured approach to verify system functionality and reliability. A sample directory containing different file types was created

to simulate real-world usage conditions. Files such as images, documents, audio, and video files were placed within the folder.

The application was executed, and the directory was selected through the graphical user interface (GUI). The program was then allowed to scan the folder, classify files based on their extensions, create necessary category folders, and move files into their respective directories.

Multiple test scenarios were considered, including mixed file types, unsupported extensions, and empty folders. After execution, the output directories were manually inspected to confirm correct file organization and ensure that files were not corrupted during movement operations.

6.3 Performance Outcome

The File Organizer application demonstrated reliable and efficient performance during testing. The system successfully scanned the selected directories, identified files based on their extensions, and categorized them into appropriate folders without errors. The execution time was observed to be minimal for typical directory sizes, indicating satisfactory processing speed.

The application consistently created required category folders and moved files accurately to their respective locations. No instances of file corruption, data loss, or unexpected behavior were detected. The GUI interaction functioned smoothly, allowing users to select folders without difficulty.

Testing with various scenarios, including mixed file types, unsupported extensions, and empty folders, confirmed that the application handled all cases gracefully. Overall, the system achieved the intended objective of automating file organization effectively and maintaining correctness across operations.

7 My learnings

Throughout the internship, I gained valuable practical experience in Python programming and real-world application development. Working on the File Organizer project enhanced my understanding of core programming concepts such as file handling, conditional logic, functions, and modular design. I developed the ability to interact with the operating system using Python libraries, which provided deeper insight into directory management and file operations.

The project also helped me learn graphical user interface (GUI) development using Tkinter, enabling me to design user-friendly applications. Additionally, I improved my problem-solving and debugging skills by identifying and resolving logical and runtime issues during implementation and testing.

This internship experience strengthened my confidence in applying theoretical knowledge to practical scenarios. The skills acquired, particularly in Python automation and GUI-based application design, will be highly beneficial for my academic projects and future career opportunities in software development and programming-related roles.

8 Future work scope

Although the File Organizer application successfully achieved its intended functionality, several enhancements can be implemented in the future to extend its capabilities. Due to time limitations, advanced features were not included in the current version but present valuable opportunities for improvement.

Future developments may include adding support for additional file categories and a wider range of file formats. The application can also be enhanced by incorporating duplicate file detection to prevent redundant storage. Implementing progress indicators or status tracking would improve user experience when organizing large directories.

Further improvements could involve scheduling automation to allow periodic file organization without manual intervention. Integration with cloud storage platforms may also be explored to enable file management across different environments. Additionally, error handling mechanisms and logging features can be strengthened to improve system robustness.

Overall, these enhancements would increase the practicality, efficiency, and scalability of the application in real-world usage scenarios.

